

Lakeflow Spark Declarative Pipeline (LDP)

Fundamentals

Outline

- Common Pipeline Settings
- Dataset Types
- Ensure Data Quality with Expectations
- Common Tasks in Production
- Change Data Capture (CDC)

Simplified Pipeline Development

Multi-file editor in Lakeflow Spark Declarative Pipelines

What's New

1. Pipeline asset browser
2. Multi-file code editor with features for step-by-step development
3. Pipeline-specific toolbars
4. Interactive DAG
5. Data previews
6. Execution insights panels
 - Easier debugging
 - Faster validation with dry run

The screenshot displays the Lakeflow Spark Declarative Pipelines interface. On the left, a 'Pipeline asset browser' (1) shows a list of assets including 'sample_trips_demo_pipeline.sql' and 'sample_zones_demo_pipeline.sql'. The central 'Multi-file code editor' (2) shows a SQL file with a toolbar (3) for running and saving. The 'Pipeline graph' (4) shows a DAG with two nodes: 'sample_trips_demo' and 'sample_zones_demo'. The 'Execution insights panels' (5) show a table of pipeline runs with columns for Name, Catalog, Schema, Type, Duration, Write, Expectations, Drop, Fail %, and Flows. A 'Data preview' (6) is shown for the 'sample_trips_demo' node.

Name	Catalog	Schema	Type	Dura...	Writ...	Expectations	Drop...	Fail %	Flows
sample_trips...	labuser1043...	default	Materialized...	7s	-	Not defined	0	-	1
sample_zon...	labuser1043...	default	Materialized...	4s	-	Not defined	0	-	1

With Spark Declarative Pipelines, you can write code using **.sql or .py files (recommended)**, or use notebooks.

As of Data & AI Summit 2025



Common Pipeline Settings

Overview

The screenshot displays the Databricks Lakeflow Spark Declarative Pipeline editor. A red box highlights the settings icon in the top-left sidebar, and a red arrow points from this icon to a detailed 'Pipeline settings' panel on the right. The main editor area shows a SQL script for creating a streaming table and a table performance section.

You can edit Lakeflow Spark Declarative Pipelines settings directly in the editor

Pipeline settings

Pipeline ID: ccb0eaa8-dfe2-4925-a36c-75e3c98b8a30
Pipeline type: ETL pipeline
Pipeline name: 1 - Developing a Simple DLT Pipeline Project
Creator: peter.styliadis@databricks.com
Run as: peter.styliadis@databricks.com

Code assets

Root folder: /Workspace/Users/p... DLT Pipeline Project
Source code: /Workspace/Users/p...s/orders_pipeline.sql
[Configure paths](#)

Default location for data assets

Default catalog: dbacademy_peter_styliadis
Default schema: default
[Edit catalog and schema](#)

Compute

Serverless

Configuration

source: /Volumes/dbacademy/ops/dbacademy_peter_styliadis
[Edit configuration](#)

Tags

[Add tags](#)

Budget

Name	Catalog	Schema	Type	Duration	Written rec...	Exp...
orders_bronze	dbacademy_peter_styliadis	1_bronze_db	Streaming table	7s	174	
orders_silver	dbacademy_peter_styliadis	2_silver_db	Streaming table	3s	174	
gold_orders_by_date	dbacademy_peter_styliadis	3_gold_db	Materialized view	5s	7	



Common Pipeline Settings



Compute



Code Assets



Configuration
(parameters)

Pipeline settings UI JSON ×

Pipeline ID: 5f9c6978-8d8b-4c21-b644-f861fb050656 🔗

Pipeline type: ETL pipeline

Pipeline name: New Pipeline 2025-04-10 10:31 ✎

Creator: peter.styliadis@databricks.com

Run as: peter.styliadis@databricks.com ✎

Code assets

Root folder 🔗: /Workspace/Users/peter.styliadis@databricks.com/ETL Expectations 🔗

Source code 🔗: /Workspace/Users/peter.styliadis@databricks.com/orders_pipeline.sql 🔗

✎ Configure paths

Default location for data assets

Default catalog 🔗: dbacademy_peter_styliadis

Default schema 🔗: default

✎ Edit catalog and schema

← Pipeline source code

Pipeline root folder
Default location for source code. Automatically appended to Python sys.path to support imports of modules and packages in this folder from source code files.

/Workspace/Users/peter com/build-data-pipeline 🔗

Source code
SQL and Python files within specified folders or provided as individual file paths are processed when the pipeline runs.

/Workspace/Users/peter com/build-data-pipeline ✕

+ Add path

The **Pipeline root folder** automatically includes all relevant files within the folder for the pipeline project (can be a Git folder)

The **source code** section references include subfolders (or files) for the pipeline (**.py, .sql, notebooks**)



Common Pipeline Settings



Compute



Code Assets



Configuration
(parameters)

A pipeline's configuration is a **map of key value pairs** that can be used to parameterize your code:

- Improve code readability/maintainability
- Reuse common parameters in multiple pipelines files

← Configuration

Key*	Value
source	/Volumes/dbacademy/ops/dbac

Add parameter

```
CREATE STREAMING TABLE bronze
SELECT *
FROM STREAM read_files(
  "${source}/orders",
  format => 'JSON');
```

In SQL reference the key's value using the **`\${key-name}`** syntax



Dataset Types Overview

How streaming tables, materialized views, and views process data



STREAMING TABLE (ST)

A table with support for streaming or incremental data processing, **only processing new data**



MATERIALIZED VIEW (MV)



VIEWS



Dataset Types Overview

STREAMING TABLE Overview

STREAMING TABLE

- Support for **batch** or **streaming** for **incremental** data processing (exactly **once processing** of the files)
- Each time an streaming table is refreshed, data added to the source tables is **appended** to the **streaming table**
- SQL code that creates streaming tables:
`CREATE OR REFRESH STREAMING TABLE`
- In SQL, you must use `FROM STREAM read_files()` to **invoke Auto Loader functionality** for incremental streaming reads and checkpoints
- File names are **guaranteed to be read only once**



Dataset Types Overview

Create a STREAMING TABLE with SQL

Create a STREAMING TABLE named **orders_bronze** from **JSON** files

```
CREATE OR REFRESH STREAMING TABLE 1_bronze_db.orders_bronze AS
```

```
SELECT
```

```
*,
```

```
current_timestamp() AS processing_time,
```

```
_metadata.file_name AS source_file
```

```
FROM STREAM read_files(
```

```
  "/Volumes/dbacademy/ops/labuser/orders",
```

```
  format => 'JSON');
```

Creates a streaming table named **orders_bronze** in the **1_bronze_db** schema

Use the SELECT clause to specify the columns to read into the streaming table from your data source

Note: The query is using the default catalog

- The **STREAM** keyword indicates to use streaming read for the source files (JSON)
- The **read_files()** function reads files under a provided location and returns the data in tabular form



Dataset Types Overview

Create a STREAMING TABLE with SQL

Create a STREAMING TABLE named **orders_silver** that reads from the **STREAMING orders_bronze** table

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver AS
```

```
SELECT
```

```
  order_id,  
  timestamp(order_timestamp) AS order_timestamp,  
  customer_id,  
  notifications
```

Specify your SQL transformations for the streaming data from the bronze table

```
FROM STREAM 1_bronze_db.orders_bronze;
```

Use **STREAM** keyword with the name of the STREAMING TABLE you are reading from to transform new rows

Note: The query is using the default catalog



Dataset Types Overview

How streaming tables, materialized views, and views process data



STREAMING TABLE (ST)

A table with support for streaming or incremental data processing, **only processing new data**



MATERIALIZED VIEW (MV)

- Records are processed as required to return accurate results for the **current data state**
- Used for data processing tasks such as:
 - Transformations
 - Aggregations
 - Pre-computing slow queries
 - Frequently used computations



VIEWS



Dataset Types Overview

MATERIALIZED VIEW Overview

MATERIALIZED VIEW

- Each time a materialized view is updated, **query results are recalculated to reflect changes in upstream datasets**
- Created and **kept up-to-date** by pipeline
- Use the **CREATE OR REFRESH MATERIALIZED VIEW** syntax
- Can be used **anywhere in your pipeline**, not just in the gold layer
- **Where applicable**, results are **incrementally refreshed for materialized views**, avoiding the need to completely rebuild the materialized view when new data arrives (**Serverless Only**)
- **Incremental refresh for materialized views** is a cost-based optimizer to power fast and efficient transformations for materialized views on Serverless compute



Dataset Types Overview

Create a MATERIALIZED VIEW with SQL

Create a MATERIALIZED VIEW named **gold_orders_by_date** that summarizes the **orders_silver** table

```
CREATE OR REFRESH MATERIALIZED VIEW 3_gold_db.gold_orders_by_date AS
```

```
SELECT
```

```
  date(order_timestamp) AS order_date,
```

```
  count(*) AS total_daily_orders
```

```
FROM 2_silver_db.orders_silver
```

```
GROUP BY date(order_timestamp);
```

Create the materialized view **gold_orders_by_date** in the **3_gold_db** schema (database)

Specify the the source table without the **STREAM** keyword to utilize the source STREAMING table
NOTE: Where possible, queries will be incrementally refreshed

Note: The query is using the default catalog



Dataset Types Overview

How streaming tables, materialized views, and views process data



STREAMING TABLE (ST)

A table with support for streaming or incremental data processing, **only processing new data**



MATERIALIZED VIEW (MV)

- Records are processed as required to return accurate results for the **current data state**
- Used for data processing tasks such:
 - Transformations
 - Aggregations
 - pre-computing slow queries
 - frequently used computations.



VIEWS

Constructs a virtual table with no physical data based on the query in your Declarative Pipelines

- TEMPORARY VIEW
- VIEW



Dataset Types Overview

TEMPORARY VIEW Overview

TEMPORARY VIEW

- Temporary views are only **persisted across the lifetime** of the pipeline and **private to the defining pipeline**
- They are **not registered** as an object to Unity Catalog
- Use the **CREATE TEMPORARY VIEW** statement
- TEMPORARY VIEWS are useful as **intermediate queries** that **are not exposed** to end users



Dataset Types Overview

VIEW Overview

VIEW

- Constructs a virtual table with no physical data based on the result-set of a SQL query in your pipelines
- They are **registered** as an object to Unity Catalog
- Use the **CREATE VIEW** statement

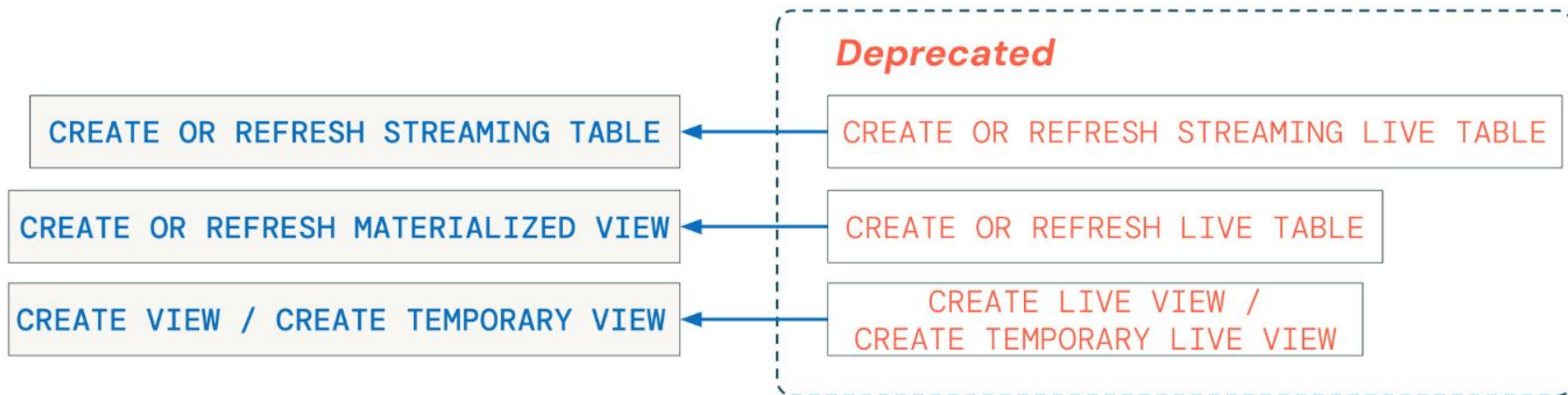
LIMITATIONS

- The pipeline must be a Unity Catalog pipeline
- Views cannot have streaming queries, or be used as a streaming source for a pipeline



Dataset Types Overview

Existing users of DLT will notice that we've evolved the names



The **semantics remain the same**, and **we'll support the old syntax** for compatibility.
Our goal is to simplify the syntax and match other systems



Dataset Types Overview

Pipeline **dependencies** are parsed automatically, code order is not required

Raw JSON -> Bronze

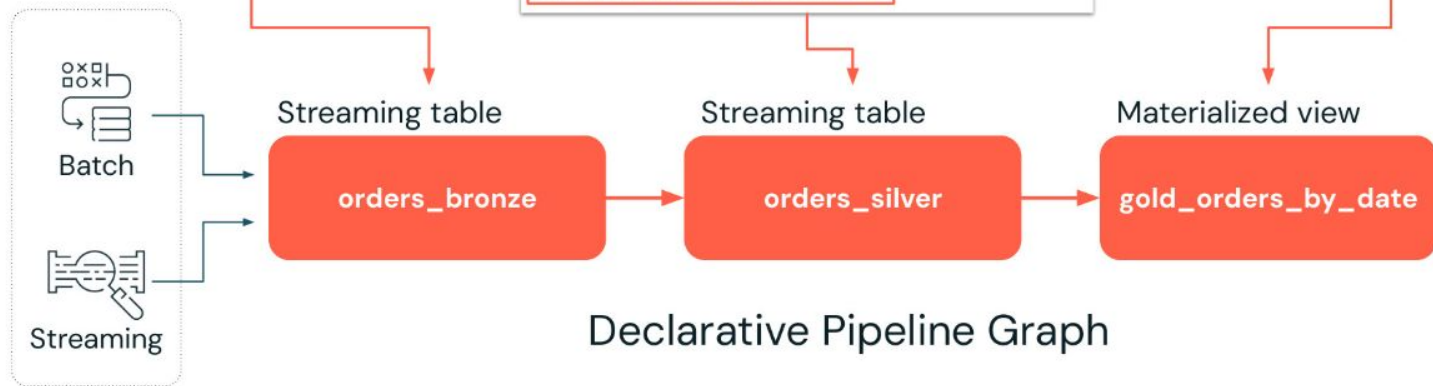
```
CREATE OR REFRESH STREAMING TABLE 1_bronze_db.orders_bronze
AS
SELECT
*,
current_timestamp() AS processing_time,
_metadata.file_name AS source_file
FROM STREAM read_files(
"/Volumes/dbacademy/ops/labuser/orders"
format => 'JSON');
```

Bronze -> Silver

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver
AS
SELECT
order_id,
timestamp(order_timestamp) AS order_timestamp,
customer_id,
notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

Silver -> Materialized View

```
CREATE OR REFRESH MATERIALIZED VIEW 3_gold_db.gold_orders_by_date
AS
SELECT
date(order_timestamp) AS order_date,
count(*) AS total_daily_orders
FROM 2_silver_db.orders_silver
GROUP BY date(order_timestamp);
```



Declarative Pipeline Graph



Ensure Data Quality with Expectations

Creating Expectations

With Lakeflow Spark Declarative Pipelines you can apply **data quality rules** to validate rows during ETL processes to gain insights into data integrity using the following

```
CONSTRAINT constraint-name EXPECT (column condition) [ON VIOLATION action]
```

Action	SQL Syntax Example	Result
WARN (default)	<code>CONSTRAINT valid_notification EXPECT (notifications IN ('Y','N'))</code>	- Invalid rows are still written to the target - Logs include counts of valid vs. invalid records and other metrics
DROP	<code>CONSTRAINT valid_date EXPECT (order_timestamp > "2021-01-01")</code> <code>ON VIOLATION DROP ROW</code>	- Invalid rows dropped from table - The count is logged alongside other metrics
FAIL	<code>CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL)</code> <code>ON VIOLATION FAIL UPDATE</code>	- Causes a failure of a single flow and does not cause other flows in your pipeline to fail. Manual intervention is required



Action on invalid record

You must specify an action to determine what happens when a record fails the validation check. The following table describes the available actions:

Action	SQL syntax	Python syntax	Result
<code>warn</code> (default)	<code>EXPECT</code>	<code>dp.expect</code>	Invalid records are written to the target.
<code>drop</code>	<code>EXPECT ... ON VIOLATION DROP ROW</code>	<code>dp.expect_or_drop</code>	Invalid records are dropped before data is written to the target. The count of dropped records is logged alongside other dataset metrics.
<code>fail</code>	<code>EXPECT ... ON VIOLATION FAIL UPDATE</code>	<code>dp.expect_or_fail</code>	Invalid records prevent the update from succeeding. Manual intervention is required before reprocessing. This expectation causes a failure of a single flow and does not cause other flows in your pipeline to fail.

You can also implement advanced logic to quarantine invalid records without failing or dropping data. See [Quarantine invalid records](#).

Ref: <https://docs.databricks.com/aws/en/ldp/expectations#action-on-invalid-record>

Syntax

Expectation decorators come after a `@dp.table()`, `@dp.materialized_view` or `@dp.temporary_view()` decorator and before a dataset definition function, as in the following example:

Python

```
from pyspark import pipelines as dp

@dp.table()
@dp.expect(description, constraint)
@dp.expect_or_drop(description, constraint)
@dp.expect_or_fail(description, constraint)
@dp.expect_all({description: constraint, ...})
@dp.expect_all_or_drop({description: constraint, ...})
@dp.expect_all_or_fail({description: constraint, ...})
def <function-name>():
    return (<query>)
```

Parameters

Parameter	Type	Description
<code>description</code>	<code>str</code>	Required. A description that identifies the constraint. Constraint descriptions must be unique for each dataset.
<code>constraint</code>	<code>str</code>	Required. The constraint clause is a SQL conditional statement that must evaluate to <code>true</code> or <code>false</code> for each record. The constraint contains the actual logic for what is being validated. When a record fails this condition, the expectation is triggered.

The `expect_all` decorators require descriptions and constraints to be passed as a `dict` of key-value pairs.

Ref: <https://docs.databricks.com/aws/en/ldp/developer/ldp-python-ref-expectations>

Ensure Data Quality with Expectations

Adding Data Quality Expectations Example

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver
(
  CONSTRAINT valid_notifications EXPECT (notifications IN ('Y', 'N')),
  CONSTRAINT valid_date EXPECT (order_timestamp > "2021-01-01") ON VIOLATION FAIL UPDATE,
  CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW
)

AS
SELECT
  Order_id,
  timestamp(order_timestamp) AS order_timestamp,
  Customer_id,
  notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

WARN in metrics

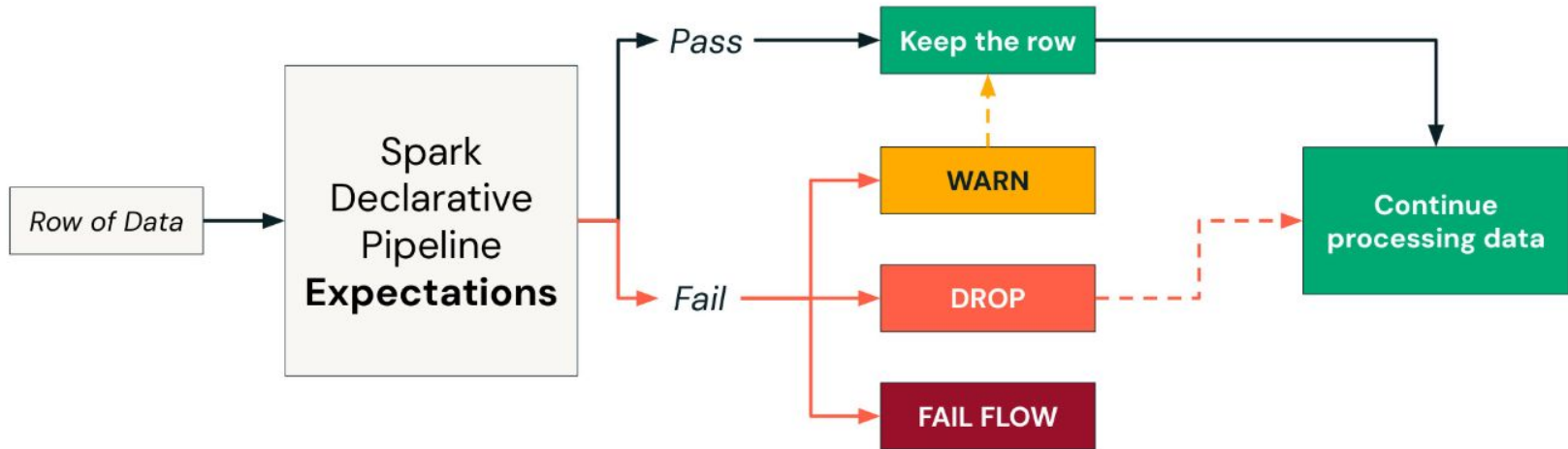
DROP the row

FAIL THE PIPELINE



Ensure Data Quality with Expectations

Actions Overview



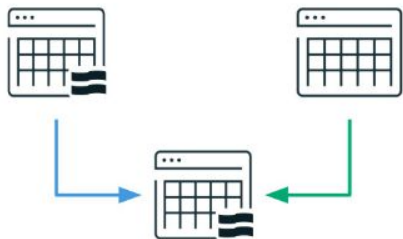
As of 2025Q2 materialized views that use **expectations** are **always fully refreshed**



Streaming Joins Overview

Stream-Snapshot Join Overview

Streaming Table to Static Table



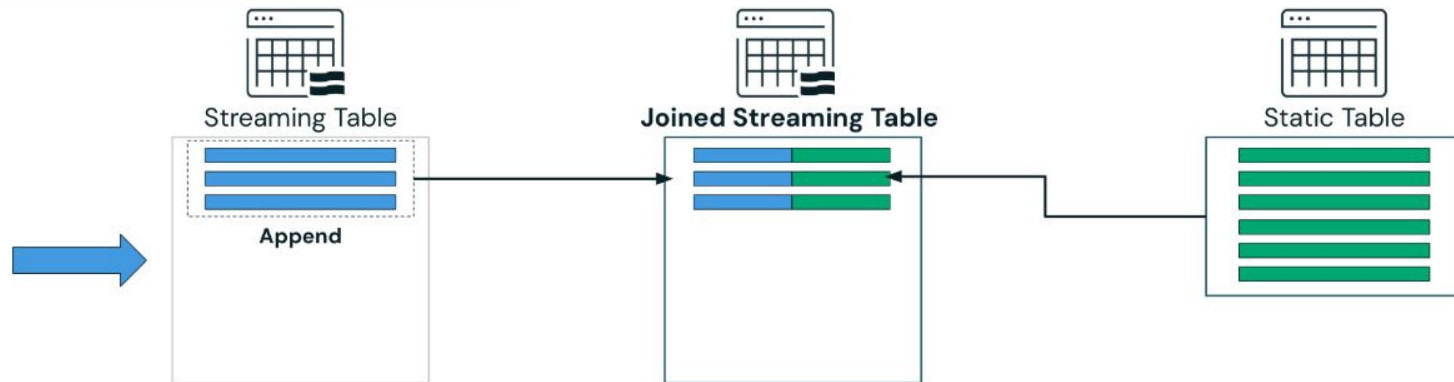
The goal is to **incrementally** join new data with a **static table**



Streaming Joins Overview

Stream-Snapshot Join Overview

Streaming Table to Static Table



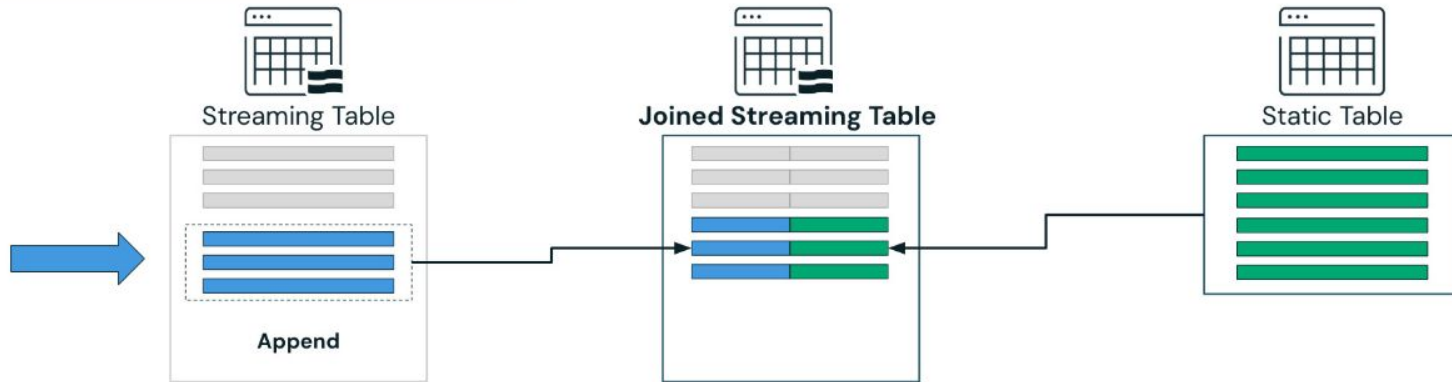
The **new data** in the streaming table **JOINS** using all of the **data in the lookup table**



Streaming Joins Overview

Stream-Snapshot Join Overview

Streaming Table to Static Table



Only the new data in the streaming table JOINS using all of the data in the lookup table

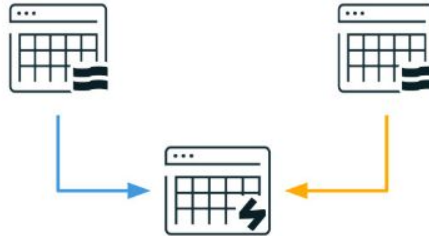


Streaming Joins Overview

Joining Streaming Tables with a Materialized View

Streaming Table to Static Table

Streaming Table to Streaming Table with a Materialized View



The goal is to take **all rows from two streaming tables** and join them together each time the pipeline is run

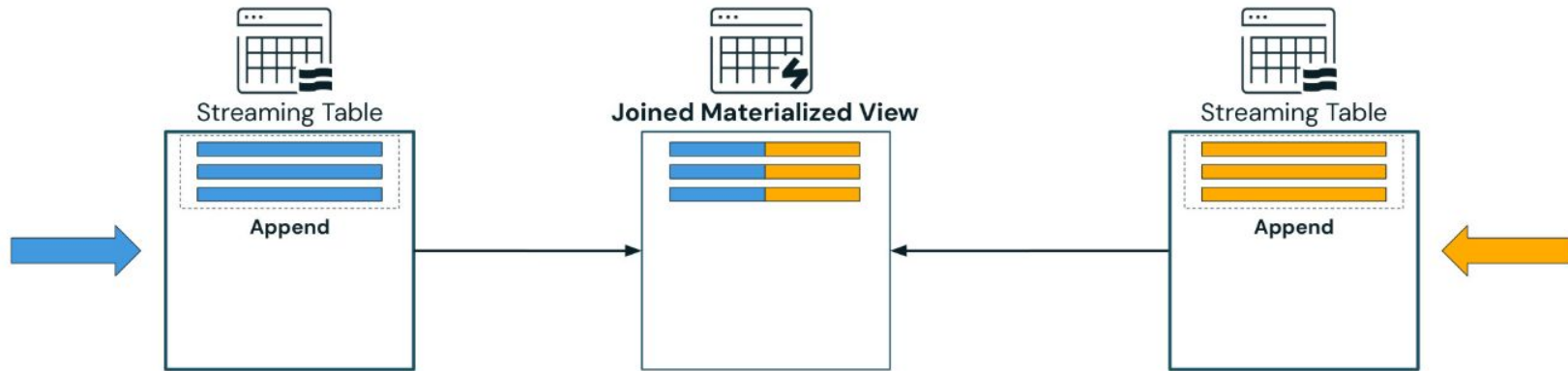


Streaming Joins Overview

Joining Streaming Tables with a Materialized View

Streaming Table to Static Table

Streaming Table to Streaming Table with a Materialized View



The materialized view **efficiently computes all the data** in the streaming tables and joins the rows.



Streaming Joins Overview

Joining Streaming Tables with a Materialized View

Streaming Table to Static Table

Streaming Table to Streaming Table with a Materialized View



As new data is added to the streaming tables, the materialized view will again **efficiently compute all the data** in the streaming tables and joins the rows



Streaming Joins Overview

Stream-Stream Joins

Outside the scope of the course

Streaming Table to Static Table

Streaming Table to Streaming Table with a Materialized View

Incremental Stream to Stream Joins

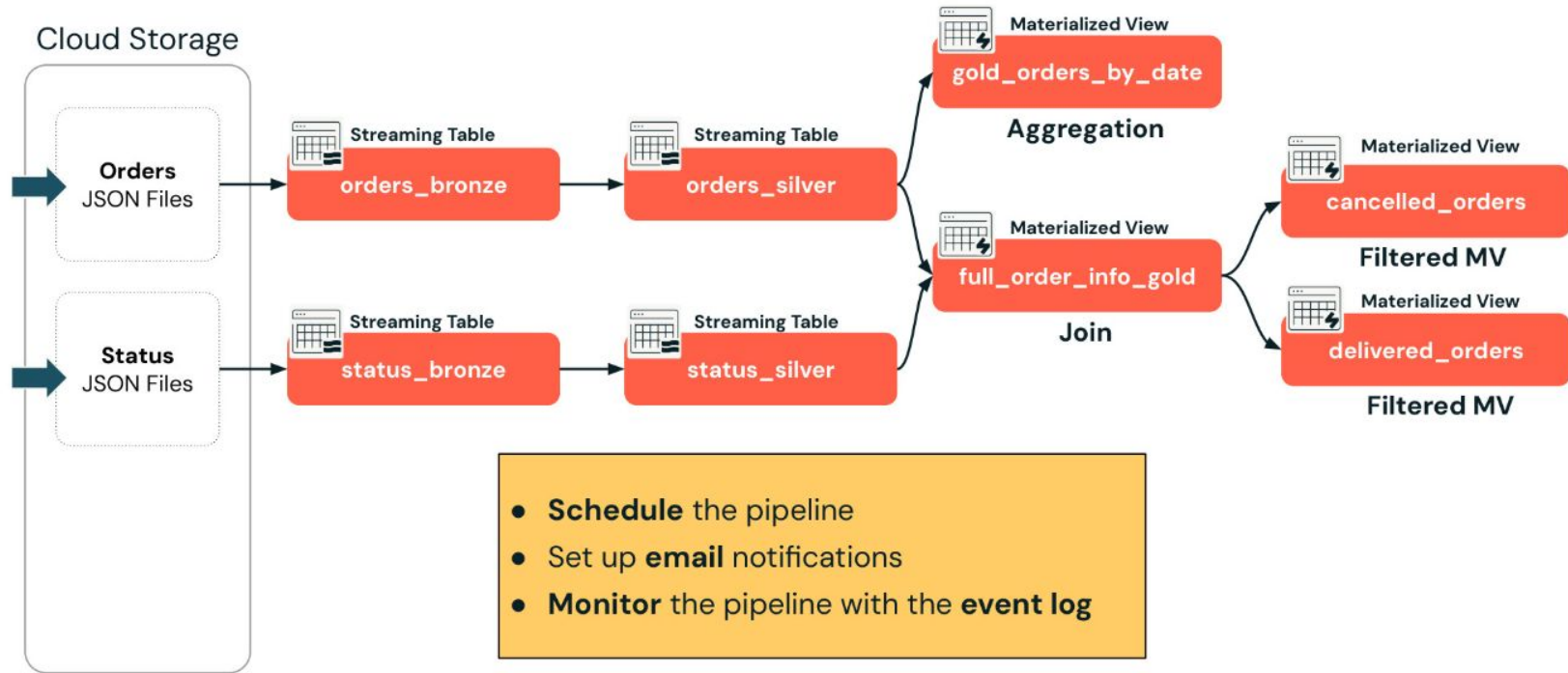


The goal is to **incrementally** join new data from two tables, **past data is not used**

[Databricks Streaming and DLT](#) (Advanced)
Course



Deploying a Pipeline to Production



Deploying a Pipeline to Production

Common Production Tasks



Scheduling



Email
Notifications



Monitor
pipelines



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Deploying a Pipeline to Production

Common Production Tasks



Scheduling



Email
Notifications



Monitor
pipelines

Triggered

- Can be triggered **manually** or run on a **schedule**
- **Refreshes** selected tables using data available at the start, then stops once the update is complete

Continuous

- **Continuously** processes new data to keep streaming tables and materialized views up to date in **near real time**
- Monitors dependencies and **updates only** when source data changes



Deploying a Pipeline to Production

Common Production Tasks



Scheduling



Email
Notifications



Monitor
pipelines

You can configure **email notifications**
when **scheduling** the pipeline

Schedule

Every at :

☐ Show cron syntax

Timezone

Performance optimization ⓘ

☐ Performance optimized [New](#)

More options ⓘ

Notifications ⓘ

☒ Start ☒ Success ☒ Failure

[+ Add](#)



Deploying a Pipeline to Production

Common Production Tasks



Scheduling



Email
Notifications



Monitor
pipelines

- The **pipeline event log** captures all key information about the pipeline:

- Audit logs
- Data Quality Checks
- Pipeline Progress
- Data Lineage



Deploying a Pipeline to Production

Querying the Declarative Pipeline Event Log

- **Publish the event log** as a **Delta table** using the **advanced settings**
 - Specify the table **location** (catalog, schema)
 - Define the **table** name

Query the pipeline event log table

```
SELECT * FROM <catalog>.<schema>.<event_log_table_name>
```



Deploying a Pipeline to Production

Querying the Declarative Pipeline Event Log

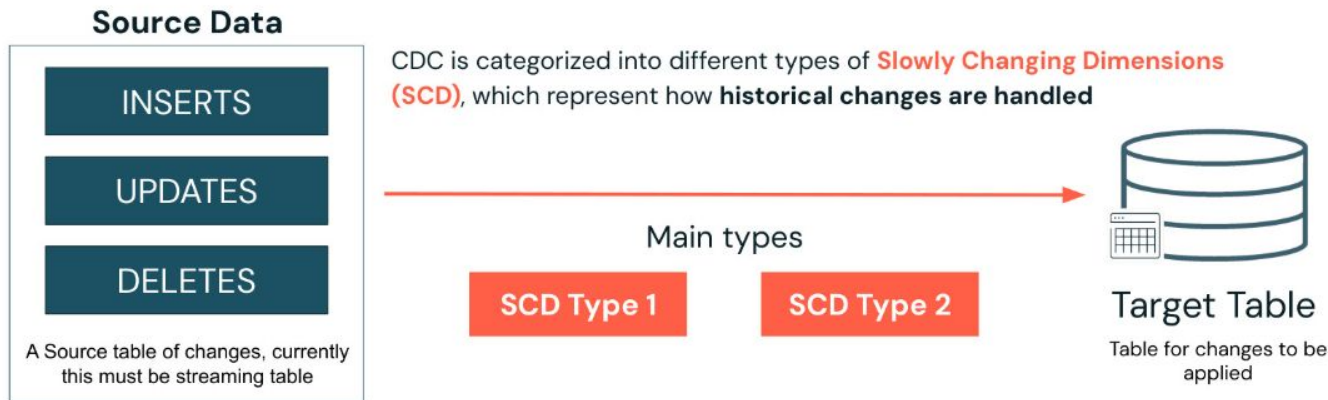
- Publish the event log as a Delta table using the advanced settings
 - Specify the table location (catalog, schema)
 - Define the table name
- (DEFAULT) The Event Log is written as a **hidden Delta table**
 - Located in the pipeline **default catalog** and **schema**
 - View the [query the event log documentation](#) to access the hidden event log



Change Data Capture (CDC) Overview

What is Change Data Capture?

- Change Data Capture (CDC) is a technique **used to track and capture changes in a data source** (such as a database, lakehouse or data warehouse).
- Then **applying those changes** into the Lakehouse or data warehouse



Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 1 (1 of 2)

SCD Type 1 (Overwrite Target with Latest Values)

- When a record updates, the **previous record is simply overwritten** by its **key** with the new value.
- When a record is deleted by its **key**, the **record is removed**
- There is **no tracking of old keys (rows)**, only the **current data** is retained.



Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 1 (1 of 2)

SCD Type 1 (Overwrite Target with Latest Values)

- When a record updates, the **previous record is simply overwritten** by its **key** with the new value.
- When a record is deleted by its **key**, the **record is removed**
- There is **no tracking of old keys (rows)**, only the **current data** is retained.

customers (*target table with all customers*)

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025



Update the **customers** table with the **new customer** information

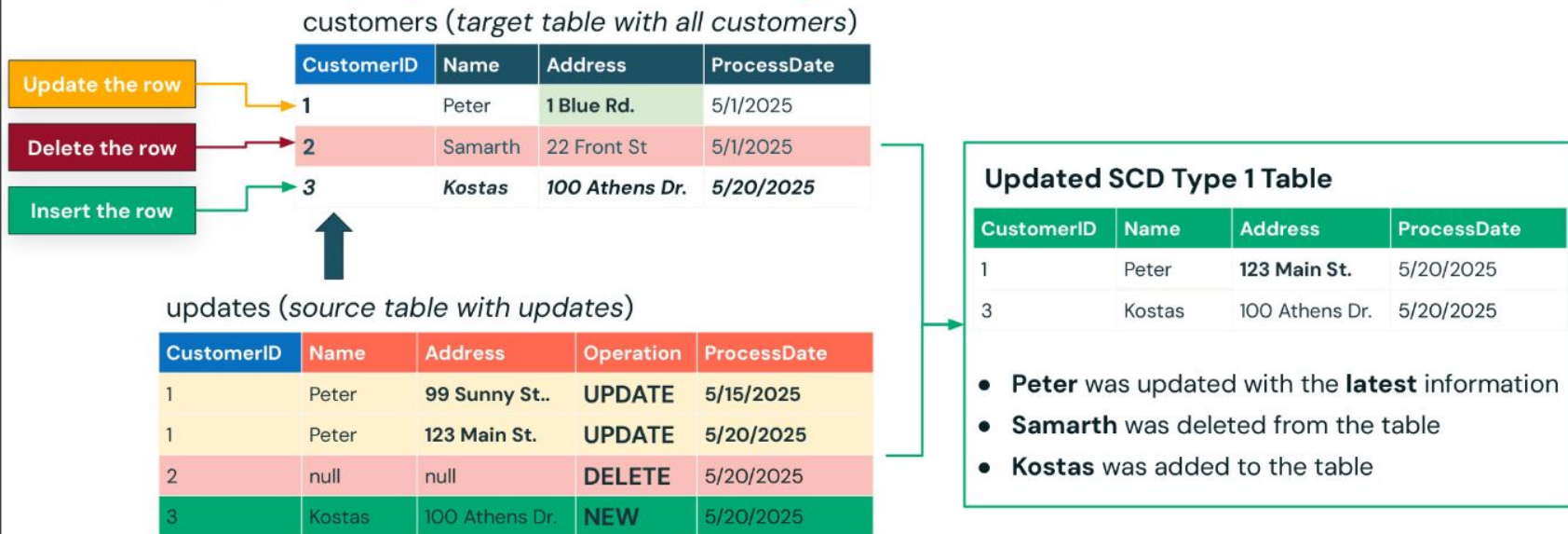
updates (*source table with updates*)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	99 Sunny St..	UPDATE	5/15/2025
1	Peter	123 Main St.	UPDATE	5/20/2025
2	null	null	DELETE	5/20/2025
3	Kostas	100 Athens Dr.	NEW	5/20/2025



Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 1 (2 of 2)



Used **SCD Type 1** when only the **current data matters** and historical values are irrelevant



Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 2 (Outside the scope of the course)

SCD Type 2 (Historical Tracking/Versioning)

- *When a record changes:*
 - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
 - When a **record is marked as deleted**, the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).



Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 2 (Outside the scope of the course)

SCD Type 2 (Historical Tracking/Versioning)

- When a record changes:
 - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
 - When a **record is marked as deleted**, the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).

Example customers target table with SCD Type 2

CustomerID	Name	Address	ProcessDate	__START_AT	__END_AT
1	Peter	123 Main St.	5/20/2025	5/20/2025	null
1	Peter	1 Blue Rd.	5/1/2025	5/1/2025	5/20/2025
2	Samarth	22 Front St	5/1/2025	5/1/2025	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025	5/20/2025	null

Columns indicate **active**
and **inactive** rows

Null values indicate the **current**
row

Non null values indicate **inactive**
(**historic**) rows



Change Data Capture (CDC) Overview

Using AUTO CDC INTO in Spark Declarative Pipelines (Formerly APPLY CHANGES INTO)

customers (target table with all customers)

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025



updates (source table with updates)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	99 Sunny St..	UPDATE	5/15/2025
1	Peter	123 Main St.	UPDATE	5/20/2025
2	Samarth	22 Front St	DELETE	5/20/2025
3	Kostas	100 Athens Dr.	NEW	5/20/2025

```
CREATE OR REFRESH STREAMING TABLE customers;
```

Create a streaming table, then use AUTO CDC to populate it

```
CREATE FLOW scd_type_1_flow AS
```

```
AUTO CDC INTO customers
```

Apply updates, inserts and deletes to the **target** table

Source records to determine **updates, deletes and inserts**

```
FROM STREAM updates
```

The column(s) that **uniquely identify a row** in the source and target tables

```
KEYS (CustomerID)
```

```
APPLY AS DELETE WHEN operation = "DELETE"
```

Specifies when a CDC event should be treated as a **DELETE** rather than an upsert.

```
SEQUENCE BY ProcessDate
```

Specifies the **logical order of CDC events** in the source data

```
COLUMNS * EXCEPT (operation)
```

Specifies a **subset of columns** to include in the **target**

```
STORED AS SCD TYPE 1;
```

Whether to store records as **SCD type 1 (default) or type 2**

INSERTs and UPDATEs are implicitly implemented using the **KEY(s)**, no coding required

Use **AUTO CDC INTO** (Formerly **APPLY CHANGE INTO**) instead of a traditional batch job with complex **MERGE INTO** syntax

Use **AUTO CDC FROM SNAPSHOT** (Public Preview as of 2025Q2) to process changes in database snapshots.



Change Data Capture (CDC) Overview

